

Foundations of Data Science for Biologists

Wrapping Up Unix

BIO 724D

2026-FEB-17

Instructors: Greg Wray and Kayla Wilhoit

Learning goals for today

Additional useful file and pipe operations

Making scripts executable

Parallel processing

Environments

Incorporating third-party software into scripts

Runninig third-party software in parallel

The find command

Introducing the `find` command

`find` locates files and directories:

- Uses a specified **search pattern** (highly flexible, many criteria)

- Searches recursively from a specified **starting directory** (computational superpower)

For every match:

- Sends the **name and relative path** to `STDOUT`

- Optionally, also performs an **action**

Key concept: recursion

Recursion: when a procedure includes the ability to invoke itself

How `find` works:

- Searches a specified directory: if it locates a directory stops and invokes itself

- Searches that directory: if it locates a directory it stops and invokes itself

- Eventually, it reaches a directory that contains no directories; it searches that directory

- Then, it goes back up one level and completes searching that directory

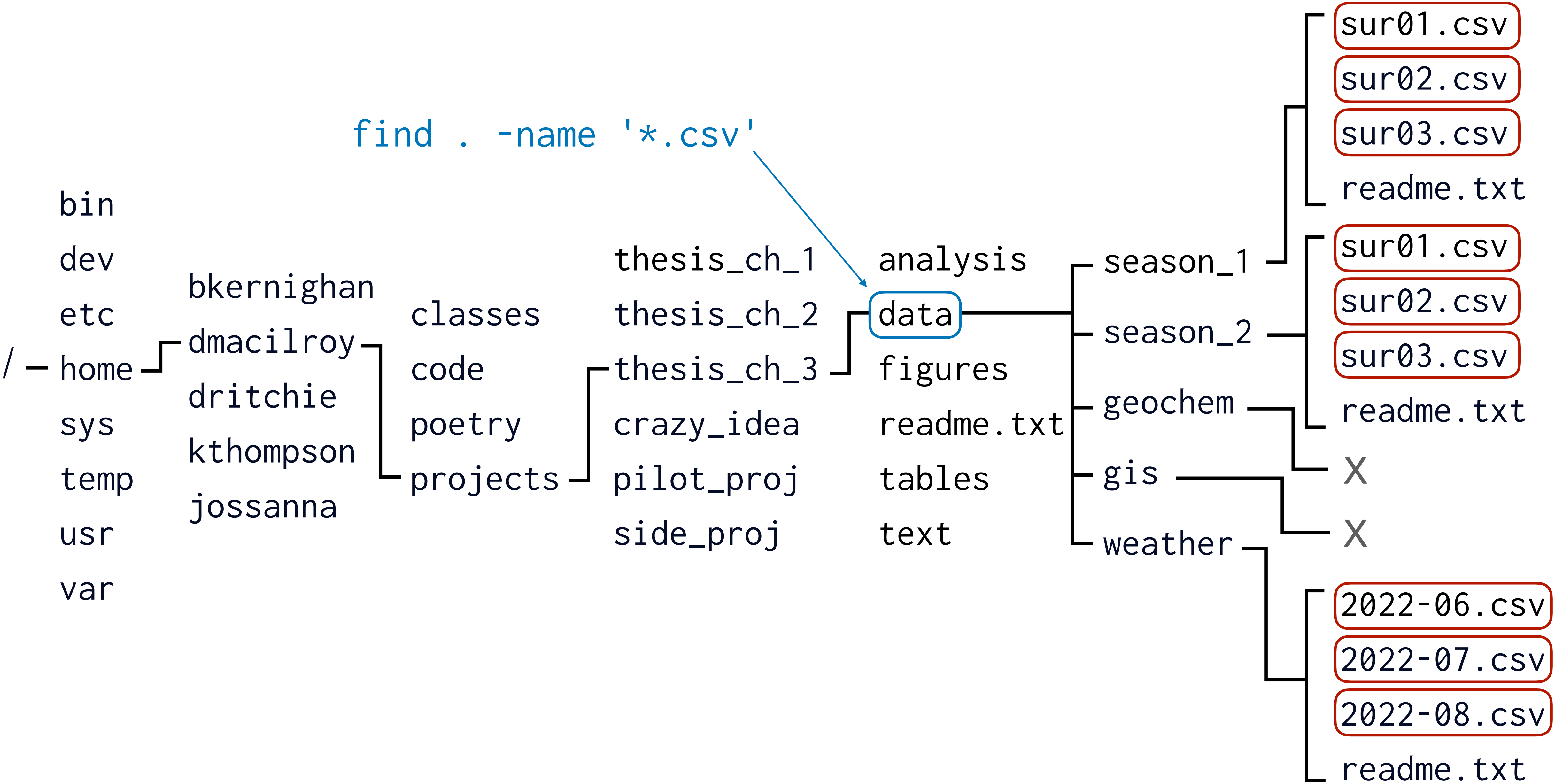
- And so forth, until the original directory has been completely searched

Called “**walking**” a directory structure: every enclosed file and directory is checked

Value of recursion: doesn't need loops or any information in advance

Walking a directory structure

```
find . -name '*.csv'
```



Specifying *where* to search

Start searching in the **present working directory**:

```
find . <pattern>
```

syntax: note the dot!

```
find . -name '*.txt'
```

example: starts searching in current location

Always use single quotes with **find**

Other **starting points** can be specified using standard paths and expansions:

```
find thesis/ <pattern>
```

from a sub-directory

```
find ~ <pattern>
```

from your home directory

```
find .. <pattern>
```

from the enclosing directory (one "up")

```
find data/ code/ <pattern>
```

from multiple locations

Specifying the *pattern*, part 1

To find files based on file **name**:

```
find . -name '*.csv'
```

list files with extension '.csv'

```
find . -name 'test_4?.txt'
```

list files that match 'test_42.txt', etc.

```
find . -iname 'test_4?.txt'
```

case-insensitive; finds 'Test_42.txt', etc.

```
find . -regex '^data'
```

searches **full paths**; does *not* find 'data.txt'

To find files based on relative **date** (note: also relative hours and many other options):

```
find . -atime -3
```

list files **accessed** in the past 3 days

```
find . -atime +365
```

list files last **accessed** more than 1 year ago

```
find . -mtime 0
```

list files **modified** today

```
find . -mtime -8
```

list files **modified** within the past week

Specifying the *pattern*, part 2

To find files based on **size** (other units and options exist):

```
find . -empty
```

list **empty** files and directories

```
find . -size -50k
```

list files smaller than 50 kilobytes

```
find . -size +100M
```

list files larger than 100 megabytes

To find files based on **type** (*many* other options exist):

```
find . -type d
```

list only directories

```
find . -executable
```

list directories and executable files

```
find . -type f
```

list only files

```
find . -type f -perm 0777
```

list only files that are read/write/exec to all

Combining patterns

Criteria can be freely combined in any order (but may not be processed in that order):

```
find . -name '*.csv' -mtime 1
```

AND is assumed with multiple criteria

Using Boolean logic:

```
find . -name '*.csv' -a -mtime 1
```

-a operator makes AND explicit (clearer)

```
find . -name '*.csv' -o -mtime 1
```

-o operator specifies OR

```
find . ! -name '*.csv'
```

! operator negates what follows

For compound conditions:

It is possible to use **\(** and **\)** to group criteria and/or force precedence

However, a pipe is often clearer and easier to debug for complex filtering operations

Specifying *how* to search

Specify the **depth** of search relative to the starting point (1 = `pwd`):

```
find . -maxdepth 1
```

limit search to the current directory

```
find . -mindepth 2
```

start searching in immediate subdirectories

Special search conditions:

```
find . -L
```

extends the search through symbolic links

```
find . -mount
```

don't search on other filesystems

```
find . -xdev
```

same as above (for compatibility)

Specifying *actions* on matches

Optionally, **find** can carry out one or more actions on files that match the search criteria:

```
find . -name '*.csv' -print
```

lists files with relative paths (default)

```
find . -name '*.csv' -ls
```

lists files with more information (= **ls -dils**)

```
find . -name '*.csv' -delete
```

deletes matching files (use with caution!!)

```
find . -name '*.csv' -fprint f
```

prints file names to file **f**

It is possible to specify any valid command to be carried out on files that match:

```
find . -type d -empty -exec rmdir {} \;
```

deletes empty directories

```
find . -type d -empty -ok rmdir {} \;
```

asks permission for each deletion

However, it is generally clearer to use a pipe with the **xargs** command (see later slides)

Useful examples of find

Display all hidden files, beginning with subdirectories

```
find . -mindepth 2 -type f -name '.*'
```

Delete all regular files that are empty from the current directory:

```
find . -maxdepth 1 -type f -empty -delete
```

Display the length all `.txt` files and sort by length:

```
find . -name '*.txt' -exec wc -l {} \; | sort -n
```

Delete all regular files that have not been accessed in >100 days, prompting for each:

```
find . -atime +100 -type f -empty -ok rm {} \;
```

The xargs command

Passing arguments in pipes

The `xargs` command is used to pass **arguments** rather than **output** in pipes

Consider the following examples:

```
ls *.txt | head
```

returns first 10 matching file names

```
ls *.txt | xargs head
```

returns first 10 lines of each matching file

First case: a single list is passed as input to `head`, which runs once

Second case: file names are passed one at a time to `head`, which runs once for each

Note: `xargs` can also pass arguments to a set of commands that run in parallel

We won't cover this, but you may encounter it in bash scripts

Combining xargs with find

A common use for `xargs` is to pass file names produced by `find` to another command

Delete temporary files that are more than a year old

```
find . -mtime +365 -a '*.tmp' | xargs rm
```

deletes based on 2 conditions

Combine the contents of files that match specific conditions

```
find . -name '*.txt' | xargs cat > all.txt
```

combines matching files into one

List the number of occurrences of substring `END` in all `.awk` files

```
find . -name '*.txt' | xargs grep -c 'END'
```

creates a count table

Making Unix scripts executable

Different contexts for using bash scripts

When you write a bash script, think about how it is likely to be used

1. Specialized for a particular project or task, you are keeping it as documentation
2. Generalized, you anticipate using it for other tasks and projects

Using specialized bash scripts

Script is specialized for a particular project or task

Store the script within the directory for the project it supports

Use the `.sh` file extension to indicate it is a bash script (not required but recommended)

To run the script, use the `sh` (or `bash`) command:

```
sh my_script.sh
```


Using generalized bash scripts

Script is generalized, you anticipate using it for other tasks

Be sure the shebang is on the first line and correctly formatted; optionally, remove `.sh`

```
chmod +x my_script  
./my_script
```

change permission to executable
run the script (`./` is usually required)

Optionally, store the script in a folder where it is accessible from anywhere:

```
mkdir ~/scripts  
mv my_script ~/scripts  
export PATH=$PATH:/home/gwray/scripts  
my_script
```

create a directory for your scripts
move the script into that directory
add the directory to your `PATH`
run the script from anywhere

File permissions and ownership

Every file and directory is assigned a set of **permissions**

Permissions determine who can **r**ead, **w**rite, and **e**xecute the file or directory

r = view contents, **w** = change contents, **x** = run (programs) or view (directories)

Every file and directory has an **owner** and a **group**. By default, you are:

Owner and group member of every file in your home directory and subdirectories

The only member of your group (you can designate others)

Permissions are managed separately for owner, group, and other

The owner can read, modify, delete, move, copy, execute, and change ownership

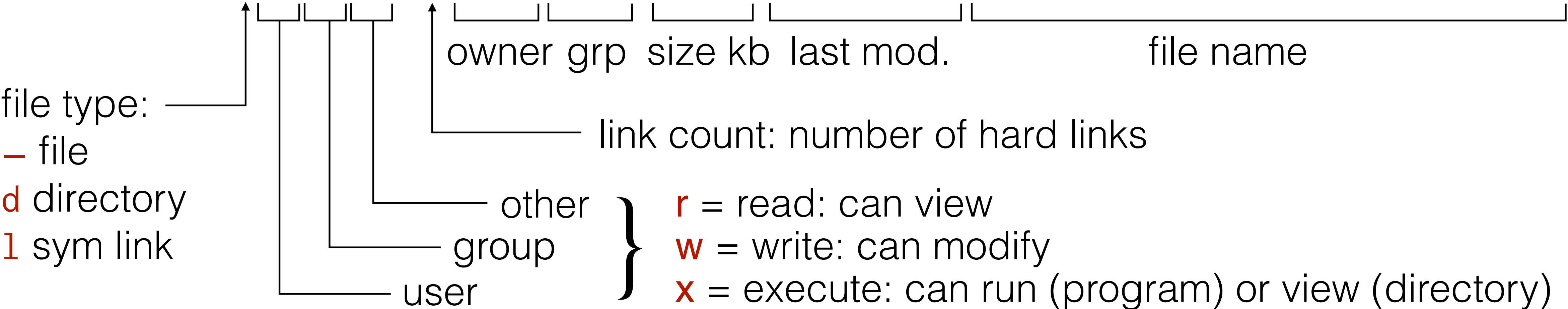
Members of the group and everyone else typically have limited or no access

The root (superuser) has complete access to every file and directory

Interpreting “long” file listings

ls -al

```
● (base) gwrap@vcm-45160:~/IOC_list$ ls -al
total 15320
drwxrwxr-x  2 gwrap gwrap    259 Jan 21 19:33 .
drwxr-x--- 15 gwrap gwrap   4096 Feb 16 11:11 ..
-rw-rw-r--  1 gwrap gwrap   5074 Jan 20 21:19 15_supp.txt
-rw-rw-r--  1 gwrap gwrap 4621459 Jan 21 15:32 df1.csv
-rw-rw-r--  1 gwrap gwrap 1280906 Jan 21 15:38 df2.csv
-rw-rw-r--  1 gwrap gwrap 1280867 Jan 21 15:43 df3.csv
-rw-rw-r--  1 gwrap gwrap 1280902 Jan 21 15:46 df4.csv
-rw-rw-r--  1 gwrap gwrap 1280882 Jan 21 15:50 df5.csv
-rw-rw-r--  1 gwrap gwrap     35 Jan 21 15:45 h1.txt
-rw-rw-r--  1 gwrap gwrap 1280949 Jan 21 16:01 IOC_14.2_clean.csv
-rw-rw-r--  1 gwrap gwrap 4621458 Jan 21 11:38 IOC_14.2.csv
-rw-rw-r--  1 gwrap gwrap     67 Jan 21 15:59 meta.txt
-rw-rw-r--  1 gwrap gwrap   1552 Jan 21 19:46 unix_data_wrangling_partI_complete.txt
-rw-rw-r--  1 gwrap gwrap    556 Jan 21 15:09 unix_data_wrangling_partI.txt
```



The chmod command

The `chmod` command is used to change permissions:

```
chmod +x my_script
```

change to executable by owner

```
chmod 700 my_script
```

alternative syntax for above

```
chmod 644 my_script
```

change to not executeable by owner

```
chmod 440 final_text.txt
```

change to read-only by owner and group

`chmod` provides fine-grained control over permissions

We won't cover all options in this class

However, it's good to be aware of this command and what it is used for

Parallel processing

A common situation: maximizing computing with limited resources

Most modern laptop computers and all compute clusters contain multiple CPUs

Most of the time, **many** of these CPUs are **idle**  

In some situations, you may want to run multiple time-consuming commands

E.g., simulations, image processing, genetic association, read mapping, alignment, finite element analysis, phylogenetic analyses, artificial intelligence, and more

The obvious solution is running jobs on multiple CPUs at once

In practice, however, it is difficult to leverage idle CPUs

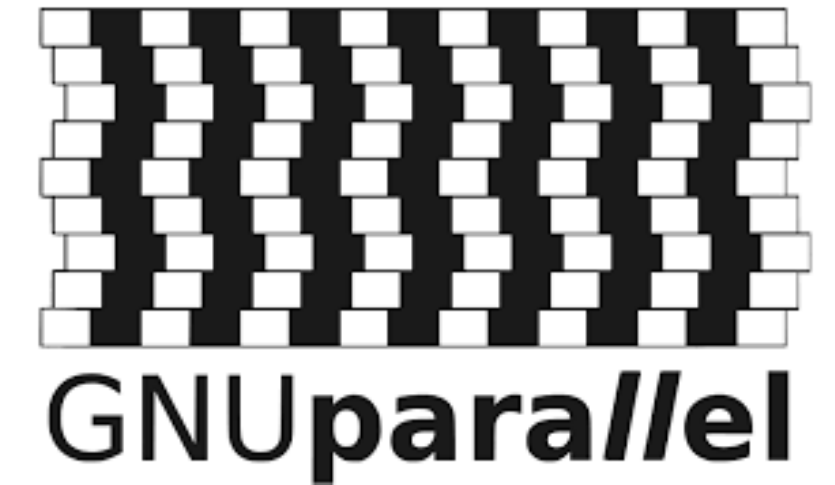
Requires scheduling software or writing code that is parallelizable



Introducing parallel

`parallel` lets you run multiple similar jobs on multiple CPUs at once

This is common on a cluster like DCC, but unusual on laptops



How it works:

`parallel` takes multiple jobs and splits them across multiple CPUs

If there are more jobs than CPUs available, the remainder are put in a queue

As individual jobs finish running, other jobs in the queue are started

`parallel` is *not* standard with most Unix / Linux distributions

It is, however, pre-installed on your VMs

To install on Windows: `sudo apt install parallel`

To install on Mac: `brew install parallel`

Passing input from the command line

The simplest approach is to pass inputs to `parallel` using the `:::` separator

This is useful for short, simple commands

Syntax:

```
parallel [options] <command> ::: <inputs>
```

each input token starts one job

Example:

```
parallel gzip ::: dir_A dir_B dir_C dir_D
```

Runs `gzip` to compress multiple directories at the same time

Passing input from a file

Another approach is to pass inputs to `parallel` from a designated file

This is useful for reproducibility and when you know the inputs in advance

Syntax:

```
parallel -a <file_name> <command>
```

each line starts one job

```
parallel <command> :::: <file_name>
```

alternate syntax; note 4 colons

Example:

```
parallel gzip :::: dir_to_compress.txt
```

Runs `gzip` in parallel on a list of directories in the file

Passing inputs through a pipe

A third approach is to pass inputs to `parallel` through a pipe

This is useful when you don't know the inputs in advance

Syntax:

```
<command> | parallel [options] <command>
```

Example:

```
find . -type d -name *_data | parallel gzip
```

Runs `gzip` to compress in parallel any directories whose names end in `_data`

Using parallel

Referencing an input as a variable:

```
parallel echo 'This is iteration {}' ::: {1..6}
```

Referencing different inputs as distinct variables:

```
parallel echo 'This is iteration {2} {1}' ::: {1..6} ::: {a..c}
```

Referencing specific columns in an input file as as distinct variables:

```
parallel --colsep ',' echo {3} {1} :::: my.csv
```

Same as above, but removing 1 header row:

```
parallel --header 1 --colsep ',' echo {3} {1}' :::: my.csv
```


Common options with parallel

By default, `parallel` runs two jobs at once, but you can change this:

```
parallel -j4 <command> ::: <inputs>
```

runs up to 4 jobs at once

```
parallel --jobs 0 <command> ::: <inputs>
```

as many at once as possible

To estimate the overall time of completion, based on the runtime of completed jobs:

```
parallel --eta <command> ::: <inputs>
```

To show progress:

```
parallel --bar <command> ::: <inputs>
```

progress bar

```
parallel --progress <command> ::: <inputs>
```

text-only version

A simple script to test the jobs option

```
#!/bin/bash

# a simple script to test the jobs option with parallel

MAX_WAIT=10                                # longest sleep time in seconds
sleep $(( $RANDOM % $MAX_WAIT ))             # random sleep time 1..10 seconds
echo "finished job $1"                      # report job completion
```


Digging into parallel

Many other capabilities make `parallel` a highly versatile scheduler:

- Commands do not have to be similar

- Inputs can be specific columns from a `.csv` file or a SQL database

- String inputs can be processed (e.g., extract substring, strip whitespace, etc.)

- File name inputs (e.g., from `find`) can be processed (e.g., strip path, extract path, etc.)

- Individual arguments and job numbers can be accessed within commands

- Output can be redirected to a pipe or to files (including formatting)

- Output from all jobs can be grouped, including preserving input order

Resources:

- Official documentation: <https://www.gnu.org/software/parallel/parallel.html#description>

- Introductory video: <https://youtube.com/playlist?list=PL284C9FF2488BC6D1>

